

04-630: Data Structures and Algorithms for Engineers

Assignment II: Toll Booth Simulation

**Carnegie
Mellon
University**
Africa

Rachel Tuyishimire

Rtuyishi

Reflection Report

Introduction

This report presents a reflection on the design and implementation of the Toll Booth Simulation, which models vehicular traffic through toll stations using a Poisson process to simulate arrival times. The simulation incorporates emergency vehicle prioritization, dynamic booth allocation based on queue lengths, and efficient processing through custom-built data structures. The primary objective was to achieve realistic system behavior while maintaining computational efficiency and scalability.

Implementation and Data Structures

The simulation is managed by a `TollboothStation` class, which oversees vehicle flow, booth activity, and data management. Vehicles arrive at exponentially distributed intervals and are stored in a custom `PriorityQueue`, which prioritizes emergency vehicles for expedited processing. Each toll booth is represented by a `Stack`, allowing it to handle one vehicle at a time, while a booth stack tracks inactive booths for dynamic allocation.

The `PriorityQueue` is implemented as a linked list. Enqueue operations incur $O(n)$ time complexity due to the need to correctly position emergency vehicles, ensuring prioritization integrity. Dequeue operations remain $O(1)$, with direct access to the front node. The linked list structure allows flexible memory management and supports dynamic insertion.

The booth `Stack` structure offers $O(1)$ operations for both push and pop, making it ideal for isolated vehicle processing. Similarly, the `BoothStack` enables constant-time operations by managing booth indices through a linked list. Vehicle data is stored in a dynamically resizing array, providing amortized $O(1)$ insertions and addressing prior fixed-size limitations.

Complexity and Testing

The overall simulation complexity is $O(n)$, where n denotes the simulation time. Vehicle generation, queue management, and booth allocation maintain optimal complexity. Following feedback, output sorting was optimized by assigning vehicle IDs at arrival

The simulation was tested across various scenarios, including edge cases such as single-booth configurations, high arrival rates, and invalid inputs. Emergency vehicle prioritization was fully implemented and verified through these tests.

Conclusion

The simulation effectively achieves its intended goals through efficient data structure design, dynamic resource allocation, and prioritized processing. Future enhancements may include implementing a heap-based priority queue for $O(\log n)$ insertion and exploring parallel booth processing for improved scalability.